KICS The Korean Institute of Communications and Information Sciences

Available online at www.sciencedirect.com
ScienceDirect

ICT Express 10 (2024) 680–691

www.elsevier.com/locate/icte

RAN Intelligent Controller (RIC): From open-source implementation to real-world validation

Mao V. Ngo[a,*], Nguyen-Bao-Long Tran[a], Hyun-Min Yoo[b], Yong-Hao Pua[a], Thanh-Long Le[a], Xian-Loong Liang[a], Binbin Chen[a], Een-Kee Hong[b], Tony Q.S. Quek[a]

[a] *Singapore University of Technology and Design (SUTD), Singapore*
[b] *Kyung Hee University, Seoul, South Korea*

Received 4 January 2024; accepted 5 February 2024
Available online 9 February 2024

Abstract

Open Radio Access Network (RAN) is an important architecture design shift for 5G and next generation telecommunications networks. With open RAN, mobile network operators would be able to mix and match multi-vendor RAN solutions as long as the solutions comply with open standards. O-RAN Alliance is the global leader in standardizing the open RAN architecture, where RAN Intelligent Controller (RIC) is positioned centrally as the brain of a RAN to manage and optimize the RAN operations. Open-source projects play a key role in accelerating the adoption of open RAN architecture, especially the use of RIC. However, the fast pace of development and the lack of documentation in open-source projects create steep learning curve for beginners. In this paper, we first provide an overview of widely used open-source RIC projects and discuss their pros and cons. We then share our first-hand experience to use RIC in our campus 5G network that consists of commercial-grade RAN solutions. In particular, we developed a suite of three RAN control applications (i.e., energy efficiency, interference management, and predictive maintenance) on an open-source RIC, and we deploy and evaluate them on a commercial-grade 5G network in a university campus. For these RIC applications, we design and evaluate different ML models based on real-world data collected from our 5G network, which we publish together with this paper. Our experimental results show that AI-based RIC applications can achieve more than 90% of accuracy in inferring the situation of the RAN for each given task. Our energy-saving RIC application can reduce 65% of energy consumption of the RAN over a simulated period of one year. Our project also validates the feasibility to interfacing an open-source RIC with existing commercial-grade 5G solutions.

© 2024 The Authors. Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Keywords: Open RAN; RIC; O-RAN; Near-RT RIC; Non-RT RIC; Use-cases; Private 5G

1. Introduction

Open RAN, as an emerging Radio Access Network (RAN) architecture design shift, emphasizes open interfaces, disaggregation, virtualization, and software-defined RAN intelligent control. Open RAN brings opportunities to accelerate innovation in mobile network operators (MNO) vendor ecosystem, by allowing smaller and new RAN vendors to join the market. O-RAN Alliance [1] is a global organization founded by MNOs that standardizes interfaces to define an open RAN architecture. With open interfaces standardized by O-RAN Alliance, not only traditional RAN vendors or MNOs can provide intelligent features to manage RAN, but third-party software and AI companies can also provide intelligent solutions to optimize RAN.

The RAN Intelligent Controller (RIC) is a key enabler of bringing intelligent radio resources management and optimization through software applications running on top of RIC, which are called xApps/rApps. RIC is categorized into two types: (i) Non-RT RIC (i.e., Non-RealTime RIC), which runs rApps, provides closed-loop non-realtime control (usually beyond 1 s) of RAN; and (ii) Near-RT RIC (i.e., Near-RealTime RIC), which runs xApps, provides closed-loop near-realtime control (usually from 10 ms to 1 s) of RAN. The Non-RT RIC (through rApps), which can be hosted as part of the SMO (services management & orchestration) platform,

* Corresponding author.
E-mail addresses: vanmao_ngo@sutd.edu.sg (M.V. Ngo), long_tran@mymail.sutd.edu.sg (N.-B.-L. Tran), yhm1620@khu.ac.kr (H.-M. Yoo), yonghao_pua@sutd.edu.sg (Y.-H. Pua), thanhlong_le@sutd.edu.sg (T.-L. Le), xianloong_liang@sutd.edu.sg (X.-L. Liang), binbin_chen@sutd.edu.sg (B. Chen), ekhong@khu.ac.kr (E.-K. Hong), tonyquek@sutd.edu.sg (T.Q.S. Quek).

Peer review under responsibility of The Korean Institute of Communications and Information Sciences (KICS).

https://doi.org/10.1016/j.icte.2024.02.001

2405-9595/© 2024 The Authors. Published by Elsevier B.V. on behalf of The Korean Institute of Communications and Information Sciences. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

provides non-real-time radio resource management services, such as policy management, network optimization, configuration management, data analytics based on AI/ML algorithms. The NearRT-RIC (through xApps) provides real-time policy enforcement, network configuration and other services.

Besides some commercial RIC-platforms (e.g., Juniper RIC [2], VMWare RIC [3], Capgemini [4], Mavernir, etc.), there are a few prominent open-source RIC-platforms that are widely used in both academic and industry, such as O-RAN Software Community (ORAN-SC) RIC (including both Non-RT RIC and Near-RT RIC). In this paper, we first provide an overview of available open-source RIC platforms, we then present our first-hand experience in developing applications on an open-source RIC platform and deploy the RIC to a commercial-grade open RAN 5G private network in our university campus.

We overcome several challenges we faced in our deployment by leveraging open-source RIC: (1) to reduce the energy consumption of our open RAN network; (2) to manage interference between neighboring cells, especially under use cases that involve mobile user equipment (UE), (3) to detect and mitigate ongoing network operation issues, and (4) to integrate commercial private 5G networks with Open-RAN architecture. We successfully leverage advantages of open RAN architecture to address these challenges. Specifically, for improving network energy efficiency, performance and reliability, we gather fine-grained RAN measurement data and develop a suite of rApps to provide accurate and granular traffic prediction at user equipment (UE)-level and cell-level, detection of network coverage and interference issues—our developed solution is able to dynamically identify emerging interference zones, while distinguishing them from non-network issues (e.g., those caused by faulty user equipment). Our experimental results show that our rApps can achieve more than 90% of accuracy in inferring the situation of the RAN for each given task. Our energy-saving rApp can reduce 65% of energy consumption of the RAN over a simulated period of one year. In addition, we also present the feasibility to interfacing an open-source RIC project with existing commercial-grade 5G solutions.

The rest of the paper is organized as follows. In Section 2, we review related works in open RAN and RAN Intelligent Controller (RIC). Section 3 presents an overview and open-source projects of Non-RT RIC and Near-RT RIC. Then, we present real use cases of Non-RT and Near-RT RIC applications on actual deployed private 5G networks in Section 4 and Section 5 respectively. Section 6 shows our experimental setup and results of the proposed applications. Finally, we summarize and conclude our work in Section 7.

2. Related work

The O-RAN Alliance was found in 2018 as an organization of MNOs, vendors, and academic institution that defines specifications for open RAN. Its goal is to shift the RAN industry toward the new paradigm for Open RAN, which are based on disaggregated RAN entities (e.g., Distributed Unit (DU), and Centralized Unit (CU), and Radio Unit (RU)), virtualized, software-based components, data-driven control with the RICs, and connected through open and well-defined interfaces, and interoperable across different RAN vendors. With open interfaces and programmable components, the RAN intelligent controllers (RICs) can run optimization with closed-loop control and orchestrate the RAN. For example, xApp on Near-RT RIC and rApp on Non-RT RIC were introduced for network slicing [5], scheduling policies [6], load balancing [7], handovers, beam mobility management, signaling storm detection, traffic steering [8], energy-saving [9], etc.

Besides open interfaces, virtualization, and intelligent, O-RAN emphasizes open-source software as one of its fundamental dimensions [10]. OpenAirInterface (OAI) [11] was established in 2014 by the non-profit organization EURECOM, and provides core networks (OAI 5G CN and EPC CN), 5G RAN stack (OAI 5G RAN), and RIC and SMO (OAI's MOSAIC5G) for intelligent control and orchestration framework. Open Networking Foundation (ONF) provides SD-RAN (i.e., μONOS as a Near-RT RIC, and a RAN simulator (CU and DU), and SDK for xApp development). And O-RAN Software Community (OSC) provides implementation of O-RAN related components (e.g., O-DU Low, O-DU High, O-CU (discontinued), Near-RT RIC, and Non-RT RIC) In Section 3, we will review all three open-source projects for RIC platforms.

Leveraging these open-source projects, many research works have been realized for different aspects. Polese et al. [12] presented a comprehensive overview of O-RAN and open-source projects for Non-RT RIC and Near-RT RIC. Johnson et al. [5] proposed a network slicing xApp called NexRAN, realized as an end-to-end Open RAN system based on open-source software. The NexRAN-xApp is deployed in OSC Near-RT RIC and integrates with open-source RAN software stack srsRAN-4G [13] on USRP. Hoffmann et al. [8] presented implementation, lessons learnt on design and evaluation of xApps (e.g., traffic steering, QoS-based resource allocation) on SD-RAN platform.

O-RAN Alliance presents a list of 25 use cases in technical reports of Working Group 1(WG1) [9,14]. These use cases can include either Non-RT RIC, or Near-RT RIC, or both. Even though these specifications defined use cases with exchanged messages between nodes, there is a room of ambiguity and freedom in implementation for xApp/rApp developers. Moreover, there is still a lack of commercially graded 5G RAN that fully supports O-RAN interfaces (i.e., O1, O2, E2 interfaces). In this paper, we address this gap by presenting our realization of rApp/xApp on an actual private 5G network deployed at SUTD by using open-source RIC projects.

3. Open-source RAN intelligent controller projects

In this section, we describe well-known open-source projects for Non-RT RIC and Near-RT RIC from O-RAN Software Community (OSC) [15], Open Networking Foundation (ONF) SD-RAN [16], and OpenAirInterface (OAI) [11].

The O-RAN Software Community (OSC) [15] is an open-source community created by O-RAN Alliance and Linux

Foundation. OSC provides reference implementation of software components for RAN, based on 3GPP standards and O-RAN Alliance specifications. OSC consists of 12 projects, where global vendors contribute to building the open-source O-RAN ecosystem. This paper focuses on the RICAPP, RIC, and NONRTRIC projects, which provide the reference implementation for O-RAN compatible RIC components [17]. The RICAPP and RIC projects are responsible for the development of Near-RT RIC and xApps, which handle RAN functions requiring real-time responses, such as mobility management and handover. The NONRTRIC project is responsible for developing the Non-RT RIC, which operates within SMO and accommodates RAN functions (e.g., RAN slicing and policy control) that do not require real-time responses. OSC's NearRT-RIC and NonRT-RIC can be deployed on both bare metal servers and virtual machines (VMs). In order to use OSC RIC, we should have knowledge of virtualization platforms such as Docker and Kubernetes.

The SD-RAN project [16] hosted by Open Networking Foundation (ONF) aims to develop controllable and programmable RAN managed by RIC. This project implements 3GPP-defined disaggregated RAN and O-RAN compliant RIC using Open Networking Operating System (ONOS). Specifically, SD-RAN leverages μ-ONOS and Google Remote Procedure Call (gRPC) framework to implement cloud-native RIC based on micro-service architecture.

FlexRIC [18] is an ultra-lean and multi-platform compatible NearRT-RIC developed by OpenAirInterfafce (OAI). This open-source platform adopts a monolithic architecture to prevent the wastage of hardware resources, such as RAM and storage, caused by microservices [19]. The E2 agent emulated in FlexRIC can interface with multiple open-source RAN platforms (e.g., OAI RAN, srsRAN) and radio access technologies. Additionally, the xApps can be developed in multiple languages, such as C/C++, and Python.

3.1. Non-RT RIC open-source platforms

For Non-RT RIC, we only see one open-source project from the O-RAN Software Community (OSC).

3.1.1. O-RAN software community—OSC

The OSC Non-RT RIC [20], as part of the O-RAN Alliance's vision, serves as an Orchestration and Automation function for intelligent management of RAN operations. Its main purpose is to support non-real-time aspects of radio resource management, optimize higher-layer procedures and policies in RAN, and provide guidance and AI/ML models to enhance the functioning of the Near-RT RIC. Non-RT RIC functions include managing services and policies, conducting RAN analytics, and training models for Near-RT RIC. The Non-RT RIC supports the **A1 interface** for providing policies, guidance, and AI/ML models from the Non-RT RIC to the Near-RT RIC [21]. The Non-RT RIC also facilitates the operation of rApps (Non-RT RIC applications) through the new **R1 interface**. The **O1 interface** is used for provisioning management services, collecting data from the network components which include Near-RT RIC, O-RAN Central Units (O-CUs) and O-RAN Distributed Units (O-DUs), and O-RAN Radio Unit (O-RUs). Such data collection is vital for enabling advanced features like network slicing and AI/ML-based network optimization. Data could be sent as an O1 notification using REST/HTTPS formatted as VES (Virtual Event Streaming) events [22]. The provisioning management services is used NETCONF protocol (specified in 3GPP TS 28.532 [23]) to create, delete, modify Managed Object Instance, and read its attributes.

The Non-RT RIC of OSC can be implemented as a logical function in SMO framework. OSC Non-RT RIC can be installed either under SMO or as a stand-alone function. The microservices comprising Non-RT RIC are deployed in *nonrtric* namespace. Each microservice is implemented as a separate container, which is hosted by Docker and orchestrated by Kubernetes using Helm as the package manager. As shown in Fig. 1, multiple microservices interact with each other to implement non-real-time management.

- **Control panel** is a web application that provides graphical user interface (GUI) for operating and managing NearRT-RIC through A1 policy.
- **A1 controller/policy management service** comprises the A1 interface function, supporting monitoring and policy configuration services using REST API.
- **A1 simulator** is a testing tool that can be used to test the function of A1 interface without the necessity of deploying Near-RT RIC.
- **rApp catalogue service** is responsible for registering and querying the rApps for interconnection with other services.
- **Information coordinator service (ICS)** is a data subscription service for interoperability of *data producer*s and *data consumer*s (subscribers of data) from different vendors. By defining the interface between data producers and data consumers, the data consumers can receive data from the data producers without the need to be aware of the specific format of the data.
- **DMaaP adapter/mediator services**, i.e., data movement as a platform (DMaaP) adapter/mediator services, is a *data producer* which acts as a data bridge between SMO and Non-RT RIC using ICS. They can retrieve information from DMaaP, which serves as a platform for data movement services within SMO (e.g., ONAP). These services can revise the data and deliver the customized data to the *data consumer* (e.g., an rApp).
- **NonRT-RIC gateway** serves as a mediator, enabling control panel to interact with A1 policy management service and ICS.
- **Helm manager** onboards and controls the microservices in NonRT-RIC as helm chart.

Pros & Cons: The NONRTRIC functions partly leverage and extend some existing infrastructure from ONAP to support non-realtime control of the RAN. Both ONAP [24] and the Non-Real-Time RIC are substantial software entities, each with unique resource requirements, environmental dependencies, and compatibility considerations with underlying

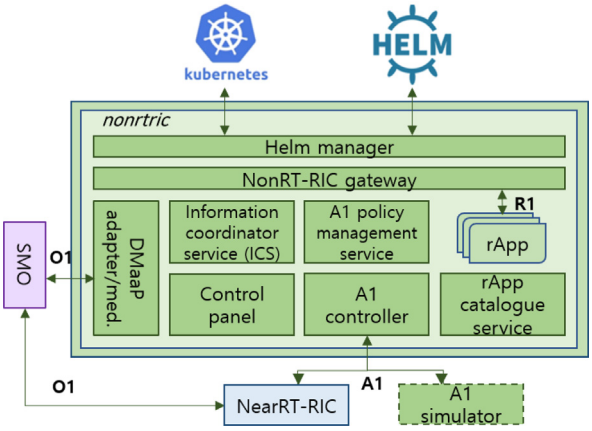


Fig. 1. ORAN-SC NonRT-RIC architecture.

hardware. Many ONAP functions are reused requiring good understanding of both infrastructure. Only deploy specific components of ONAP and ORAN for each use cases for modularized testing.

3.2. Near-RT RIC open-source platforms

For Near-RT RIC, there are three open-source platforms: OSC's Near-RT RIC, FlexRIC from OAI, and SD-RAN from ONF. We describe each one and its pros and cons in the following subsections:

3.2.1. O-RAN Software Community—OSC

The OSC Near-RT RIC leverages virtualization technologies to enable efficient resource management, service provision, and automation. The microservices in Near-RT RIC can also be deployed in a Kubernetes infrastructure. As shown in Fig. 2, ORAN-SC's Near-RT RIC provides three namespaces: *ricinfra*, *ricplt*, and *ricxapp*. The *ricinfra* handles functions responsible for the interface between Helm, Kubernetes, and RIC elements. The *ricplt* deploys functions for interactions between O-RAN elements and database systems. The users can deploy xApps in *ricxapp* using xApp descriptor. The services in these namespaces are offered through HTTP, SCTP, or TCP protocols. The following microservices are deployed in *ricplt* by default:

- **Interface mediators** support user to interact with O-RAN interfaces (i.e., O1, A1 and E2).
- **E2 termination** supports the connection between NearRT-RIC and RAN nodes.
- **E2 manager** can control the overall connections via E2 interface and routing manager.
- **Routing manager** is responsible for routing of messages within the RIC platform with the RIC message router (RMR) library.
- **Subscription manager** enables xApp to subscribe services from RAN nodes. At the time of this writing, O-RAN specification includes four services: report, insert, control, and policy.

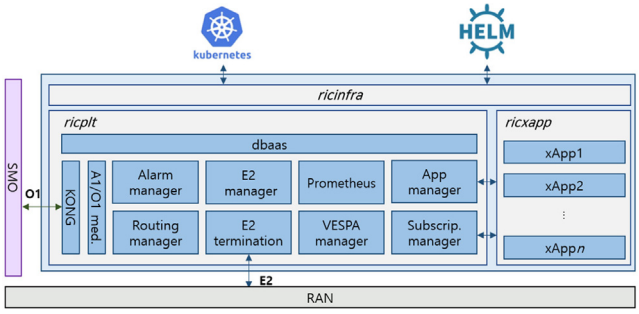


Fig. 2. ORAN-SC NearRT-RIC elements and interconnections.

- **App manager** is a function that can be used to deploy xApps in *ricxapp* namespace.
- **Alarm manager** handles the alarm coming from O-RAN interfaces, RMR, and command from administrator.
- **VESPA manager**, i.e., virtual network function (VNF) event stream (VES) Prometheus adapter manager, monitors event stream of VNFs in NearRT-RIC and collects statistics and logging using Prometheus.
- **Prometheus** is a native data collection and monitoring platform for Kubernetes that captures and processes the performance metrics of xApp and RIC elements.
- **KONG** is an API gateway for monitoring and managing the microservices in NearRT-RIC architecture.
- **dbass** provides database as a service to NearRT-RIC platform. OSC chooses the Redis as the database technology.

Pros & Cons: OSC's Near-RT RIC has been constantly updated via new releases, and many xApps have been introduced (e.g., anomaly detection, KPM monitoring, QoE predictor, traffic steering) while these xApps remain at the basic level of RAN control where they were first released. Compared to other Near-RT RIC open-source projects, OSC Near-RT RIC is completely O-RAN compliant, comprehensive for large scale deployment. However, it requires more resources to deploy due to fine-grained microservices and running on Kubernetes cluster. Also, new users need a basic knowledge of container and kubernetes to host, onboard, and run an xApp. Similar to other open-source project, if we want to use OSC Near-RT RIC to an actual 5G RAN an integration effort is needed because of two reasons: (i) at the time of writing, not many 5G RANs fully support E2 interface defined in O-RAN specifications, (ii) there is a mismatch in supported versions of E2 service models between the RAN-E2 node and the Near-RT RIC platform.

3.3. FlexRIC—OAI

FlexRIC adopts a software development kit (SDK) approach instead of network application program interface (API) approach to achieve a simple yet complete RIC architecture [25]. Fig. 3 shows the architecture of internal components of FlexRIC provided by OAI. For the purpose of achieving low latency, the xApps in FlexRIC use internal applications (iApps) and E42 protocol [26] for direct association with RAN functions. Also, it uses SWIG [27] as an interface compiler

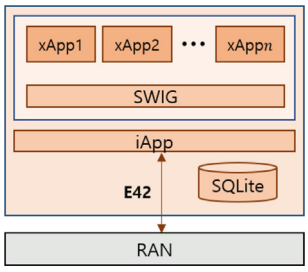


Fig. 3. FlexRIC architecture.

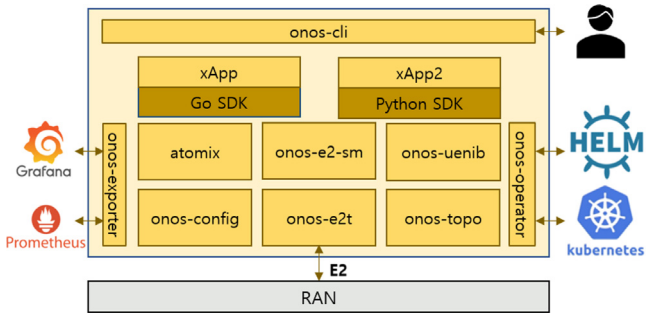


Fig. 4. SD-RAN Near-RT RIC architecture.

to enable the multi-language xApps. FlexRIC's SDK is implemented in C, yet it enables the development of xApps in different languages through SWIG. The xApps manage data using a lightweight SQLite database. Moreover, FlexRIC follows the "zero-overhead principle", i.e., this platform avoids using containers (as well as Kubernetes) to prevent unnecessary resource consumption.

Pros & Cons: FlexRIC has significantly less source code than ORAN-SC RIC, making it easier for development of various use cases. However, due to the monolithic implementation without the use of containers and Kubernetes, FlexRIC exhibits limited scalability and load management capability. Therefore, efficiently distributing resources for a large number of clients and providing automated management are challenging. Furthermore, as issues arising from a single component can have an impact on the entire system, the resilience to error occurrences is diminished. Nevertheless, thanks to the ultra-lean architecture, it can provide low-latency, making it beneficial for networks with a small number of critical clients, such as private 5G. In Section 6.2, we present a use case study where we apply FlexRIC to a commercial private 5G RAN.

3.4. SD-RAN—ONF

The SD-RAN project [16] supports an open interface to be compatible with disaggregated RAN equipment from various vendors and self-developed RAN simulator [28]. To establish connectivity between these RAN elements and the SD-RAN's Near-RT RIC, an *E2 agent* is imported into a central unit (CU) or distributed unit (DU) (in short called E2-node). Through this *E2 agent*, the E2-node and RIC communicate via E2 messages. When the E2-node needs to report telemetry information, it sends an *indication messages* to the RIC platform, which subsequently forwards them to the appropriate xApp. Conversely, if an xApp wishes to issue commands to the CU, xApp transmits a *control messages*.

The SD-RAN NearRT-RIC platform includes SDKs to facilitate xApp development, primarily offered in the Go and Python languages. These SDKs allow xApps to interact with microservices in NearRT-RIC. Similar to the OSC RIC, SD-RAN's NearRT-RIC is designed to run on a Kubernetes infrastructure and uses Helm to manage and install Near-RT RIC's packages. These packages are configured to run the xApps and all the Near-RT RIC elements in Fig. 4. The details on the SD-RAN RIC elements are as follows:

- **onos-e2t** is a proxy for E2 interface. It receives/sends E2-AP messages from/to RAN nodes.
- **onos-e2-sm** encapsulates service models into E2-AP messages for transmission to RAN, or decapsulates received E2-AP messages.
- **onos-topo** is a database that contains both topology and cell information.
- **onos-uenib** stores detailed information about the UE and connected cell.
- **atomix** is a framework for controlling distributed databases in Kubernetes-based RIC.
- **onos-cli** is an interface through which RIC elements and users can interact via the command line.
- **onos-operator** is a Kubernetes operator responsible for monitoring RIC elements, allocating resources as needed, and providing configuration when the RIC elements are deployed.
- **onos-exporter** provides GUI services for exposed data generated by onos-e2t or xApps. This element is connected to Prometheus and Grafana, and it is used to visualize stored metrics.
- **onos-config** holds configuration files for RIC elements and xApps.

Pros & Cons: At the time of this writing, the SD-RAN platform has developed six xApps: (i) the Physical Cell Identity (onos-pci); (ii) the Key Performance Indicator Monitoring (onos-kpimon); (iii) the Mobile Handover (onos-mho); (iv) the Mobility Load Balancing (onos-mlb); (v) the RAN Slice Management (onos-rsm); and (vi) the Traffic Steering (rimedo-ts). These xApps support custom logic that can be used to perform network-wide management. To test the functionalities of these xApps, the SD-RAN platform provides a RAN simulator. The RAN simulator can emulate up to approximately 1000 RANs and 100K user equipment, and describe the simulation environment using YAML files. However, the SD-RAN RAN simulator does not support physical-layer processing and complex channel models. In addition, it does not consider user plane information (e.g., throughput, physical resource blocks (PRB)). To improve the capability of SD-RAN platform, we need to implement practical channel model and user plane of the RAN simulator to thoroughly validate the performance of xApp.

4. Non-RT RIC applications for a private 5G network deployment

4.1. Energy saving and interference management applications

High energy consumption with densely deployed radio units (RUs) in 5G costs lot of Operation Expenditure (OPEX) of MNO. In addition, to meet the target of green networks of MNO, energy efficiency is not just a discussed topic but we need to evaluate proposed solutions in the real deployment and see the actual benefit of saving energy in the actual 5G deployment. Besides saving energy, interference management is an intrinsic and widely observed issue in real-life deployment because of scarce spectrum resources.

In this section, we describe a use case of using NonRT-RIC to save energy and mitigate interference in a private 5G network while maintaining quality of service (QoS) in an actual 5G private network deployed in Singapore University of Technology and Design (SUTD) campus. The 5G private network cover both indoor and outdoor environment. In particular, we setup an O-RAN-SC NonRT-RIC to collect key performance metrics (KPM) specific to individual user equipment (UE) under different use scenarios. Based on which, we developed 4 rApps that work together to improve network energy efficiency and reduce inter-cell interference.

Our SUTD 5G network consists of multiple indoor cells and one outdoor cell, operating at the same N78 band. The outdoor cell consists of 2 RRUs that jointly cover a drone arena and the corridor areas of multiple floors (i.e., levels 4, 5, and 6). The indoor cells cover our lab at level 3 and two classrooms at level 5. The level 5 indoor cell overlaps with the outdoor cell at level 5's corridor area, resulting in an interference zone there. We observed two main drawbacks in the network operation since the network was commissioned in summer 2022:

1. Our O-RUs are always on using full transmission power 24/7, though there is no 5G traffic at night time or during the weekend. Even during working hours, the 5G traffic demand varies significantly across different floors and different hours of day.
2. The inter-cell interference at the level 5 corridor causes significant performance degradation in some use cases, including one that involves patrolling robots.

We developed a suite of rApps to address these two drawbacks. Firstly, while we collect the Key Performance Metrics (KPM) of individual UEs from the RAN, they do not directly identify the location of UEs. Specifically, the GPS signal of UEs is weak in corridors of the building. Our first machine learning (ML) based *Localization-rApp* estimates the UE location (i.e., which floor the UE is) based on collected radio metrics from the RAN. Our second ML-based UE-level Traffic-Prediction (TP) rApp (or *UE-TP-rApp* for short) predicts throughput of UE based on its recent radio signal metrics (e.g., RSSI, RSRP, SINR, PRB usage, and throughput). These two rApps provide inputs for another *cell-TP-rApp* to predict traffic throughput demand of a cell. Three described rApps facilitate as inputs for making decisions on following tasks:

- Energy Saving rApp (*ES-rApp*) that turns off/on an O-RU depending on the predicted 5G traffic demand of different cells at different time.
- Interference Management *(IM-rApp)* that monitors UEs under interference zone and makes decisions to reduce transmission power of neighboring cells' O-RUs.

As a result, we can jointly reduce interference, save energy, and increase UE throughput. Specifically, the IM-rApp gets input from the Localization-rApp about the location of UEs around the interference zone, inputs from the UE-TP-rApp about each UE's traffic (e.g., a robot may need to stream a 4 K video). Based on these inputs, it controls transmission power of neighboring O-RUs.

4.2. Predictive maintenance application

Normally, the RAN can identify active RRUs by heartbeat signals periodically sent from EMS to each RRU. However, if the RRU is still actively sending heartbeat via M-plane, but the functionality of RRUs can be disruptive (e.g., some antenna loses its physical connection, so the transmission at the last mile is dropped). The EMS still sees the RRUs as normal, but it is actually malformed.

Instead, with collected measurements from KPM of UE-level fed into the Non-RT RIC, a Predictive Maintenance *(PM)-rApp* can detect how many RRUs are working. In this use-case, we construct a dataset of the KPM metrics of UE-level under different numbers of active RRUs, e.g., 1 RRU on, 2 RRUs on, etc. We then train an ML model to classify these cases. Based on the output, we can send an alert signal to the maintenance team to check any issue if there is a suspected malformed RRU.

We present our implementation, and experimentation of the described Non-RT RIC applications in Section 6.1.

5. Near-RT RIC applications for a private 5G network deployment

Commercial 5G RAN vendors typically release new updates slower than the speed of standardization organizations because vendors take time to implement and validate properly before releasing them to customers. To speed up the adoption of O-RAN architecture with the RIC platform, we leverage the open-source FlexRIC E2-agent [29] and integrate it with a commercial 5G RAN to convert a commercial 5G private RAN to an O-RAN-compliant E2-node. More specifically, we describe how to expose key performance measurements (KPM) metrics from the private 5G RAN to *KPM-xApp* deployed at Near-RT RIC via E2 interface (defined in O-RAN WG3) [30]. Fig. 5 shows our implementation with E2 agent as a proxy between Near-RT RIC xApps and commercial private 5G RAN.

5.1. KPM-xApp

KPM-xApp is responsible for collecting the KPMs from RAN to the Near-RT RIC platform via E2 interface. It stores

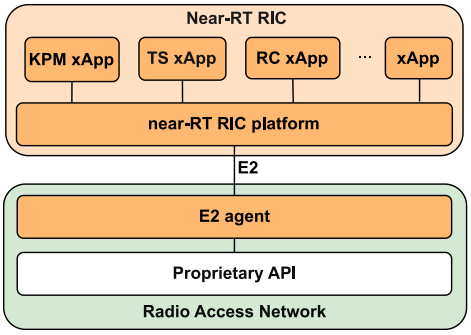


Fig. 5. Near-RT RIC xApps with a proxy E2 agent to a commercial private 5G RAN.

the data in the database in Near-RT RIC platform, which is an InfuxDB pod in OSC Near-RT RIC or SQLite database in FlexRIC. The O-RAN specifications define the E2 service models (E2-SM) with a set of metrics as a baseline for the O-RAN vendor to follow. The stored KPM metrics can be used for feature selection [31] and further xApp development such as anomaly detection [32], traffic prediction [33] or traffic steering [34].

5.2. TS-xApp

TS-xApp receives a high-level A1-policy from the A1 interface from Non-RT RIC and enforces it to the E2 node. TS-xApp can make decisions fast and increase the stability of the connection due to the requirement of 10 ms to 1 s latency of E2 interface [35]. Take the energy-saving use case as an example, ES-rApp can enforce an A1 policy that turns on/off some of the RRUs or the whole cell to save energy when some conditions are met. TS-xApp can leverage the data collected from KPM xApp to predict the traffic demand of cells and UEs, then steer the traffic to another cell to avoid disruption in user traffic before turning off RRUs for saving energy.

6. Experimental evaluation

6.1. Non-RT RIC applications for private 5G networks

We present the architecture of our implementation of xApp/rApp based on an open-source RIC platform (i.e., ORAN-SC NearRT-RIC and ORAN-SC NonRT-RIC) in Fig. 6. For the:

1. Key Performance Metrics (KPM) of individual UEs from the RAN are collected via the *VES Collector*. KPM data is streamed into the Influx Time-series database to be processed for rApp.
2. *Localization-rApp* is to estimate UE location based on radio metrics. The output of the rApp is just a floor indication of the UE instead of exact GPS-location of the UE. So, we treat this rApp as a classifier to output which floor level of UE.
3. *UE-TP-rApp* is a UE-level throughput prediction rApp that predicts the traffic throughput of UE based on its recent measured radio signal metrics (e.g., RSSI, RSRP, CQI, PRB usage, and throughput).

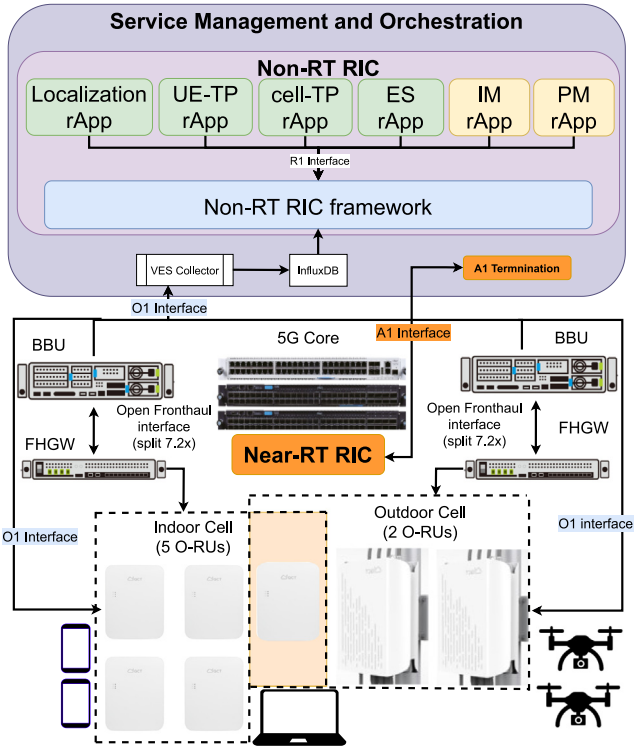


Fig. 6. System architecture of Non-RT RIC rApps with the private 5G network.

4. *Cell-TP-rAPP* is a cell-level throughput prediction rApp that predicts the traffic demand of a cell based on output from Localization rApp and UE-TP rApp.
5. *ES-rApp* is Energy Saving rAPP that makes decisions to turn off/on an O-RU depending on the predicted 5G traffic demand of different cells at different times.
6. *IM-rApp* is Interference Management rApp that monitors UEs under the interference zone and makes decisions to reduce the transmission power of neighboring O-RUs.
7. *PM-rApp* is a Predictive Maintenance rApp that monitors the number of actual active RUs under a cell, and ensures there are no malformed RUs that could be under attack or physically lose their connection. The PM-rApp predicts a number of actual active RUs based on real collected measurement feedback from UEs, so it reflects the true condition of the active RUs.

6.1.1. Data collection and Pre-processing

In this section, we will present our collected traces from actual deployment, and implement a simple AI model to either: predict traffic of UE, or detect anomaly events. The output of the AI model is an input for downstream tasks: traffic steering or adaptive resource allocation.

Data collection: We collected KPI metrics /traces from mobile devices including *RSRP, RSRQ, SINR, PDSCH PRBs, PDSCH MCS, PUSCH MCS* and *DL throughput*. These metrics can be reported to rApp for detecting real-time issues of the networks. Table 1 shows our collected traces from different

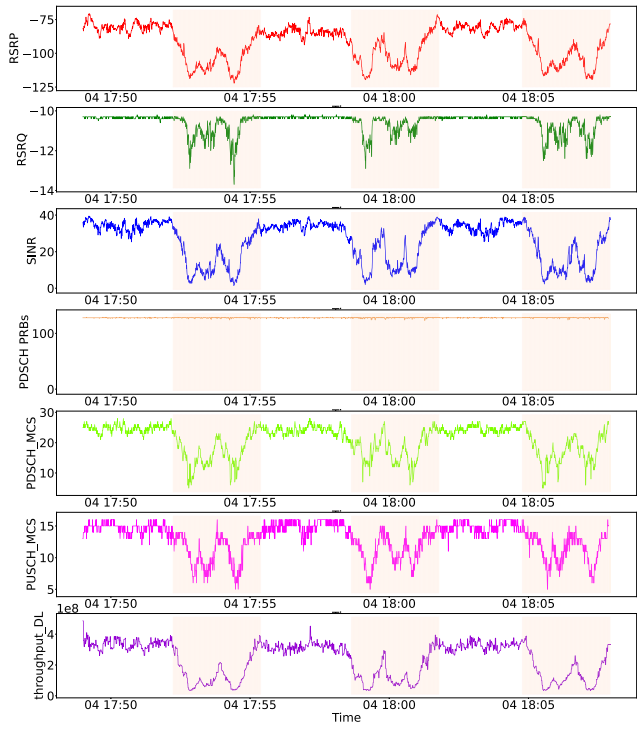


Fig. 7. Exemplary traces of scenario 3—Level 6, 2 outdoor RRU active. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this dataset.).

Table 1
Data collection: scenarios and description.

Scenarios	Duration	#Samples
1. Level 4, 2 outdoor RRUs active	24 min 58 s	2327
2. Level 5, 2 outdoor RRUs active	19 min 19 s	2031
3. Level 6, 2 outdoor RRUs active	19 min 9 s	2156
4. Level 6, 1 outdoor RRU active	18 min 56 s	2222

configurations in building 2, SUTD. In each trace, a human carries a 5G Samsung S22 Ultra smartphone UE (i.e., a mobile testing tool) and walks along the corridor of each floor for 3 rounds in a continuous manner. The KPI metrics are collected at a sampling rate of 2 Hz (i.e., 2 samples per second). As shown in Table 1, even though the number of samples is not as large as the published dataset provided in OSC RIC [32]. The total duration of our collected dataset is around 20 min which is longer than the one provided by OSC RIC. In addition, our collected dataset is based on real 5G UE with an actual 5G network, which is lacking in the O-RAN research community. Fig. 7 shows an exemplary dataset of the UE collected at level 6 when 2 outdoor RRUs are working. The orange shaded areas denote the interference area when the UE moves in the interference zone between the indoor cell and the outdoor cell. The human walked 3 rounds, so we can see 3 orange shaded areas. We publish our dataset with detailed descriptions at https://github.com/FCCLab/sutd_5g_dataset_2023.

Pre-processing step: We first drop the rows that contain null value and then we calculate pair-wise cross-correlation among these 7 KPM features to select no-redundant features for training ML models. We set a threshold of 0.98 and see all the 7 collected KPMs are non-correlation (i.e., the highest correlated pair is SINR and RSRP with 0.973 of correlation score).

As the range of different collected metrics are very different, we then normalize the data set to a range of [0,1]. After that, we split the collected data into training and test sets with a ratio of 70:30.

In this paper, we implement and evaluate different ML models for our rApps, including both instance-based input and sequence-based input. For instance-based input, the models only consider a single incoming sample and infer the output during inference. On the other hand, for sequence-based input, the ML models get a recent window of samples as input, so they can capture a temporal relation of the collected metrics as well. However, the latter approach requires constructing a bigger model to capture the larger input. In our dataset, we use a window of 10 samples, which is equivalent to 5 s in length.

6.1.2. Experimental setup

For different rApps described in Section 6.1, we design different ML models including both classical ML models (i.e., Support Vector Machines (SVM), Random Forest (RF), and XGBoost) and deep learning (DL) models (i.e., Linear classifier, linear regression, Deep Neural Networks (DNN), and Long Short-Term Memory (LSTM) [34]), for both instance-based and sequence-based inputs. The DL models for all the rApps are trained with 20 epochs and 0.001 learning rate. Table 2 describes the detailed setup of different ML models.

The selection of ML models are based on prior works. Kavehmadavani et al. [34] proposed utilizing a DL approach, which is to train LSTM model at non-RT RIC, for traffic prediction due to its capabilities in handling long time-series flow data. RF was used as the classifier model in the first release of anomaly detection xApp [36].

The *Localization-rApp* is a multi-classes classifier ML model that receives an input either an instance or a sequence of instances, and generates an output of floor level where the UE is located. We annotate floor level [4, 5, and 6] as [0,1, and 2] labels, respectively. Both DNN and LSTM are trained using cross-entropy loss function. DNN is trained with Adam optimizer and MultiStepLR scheduler where LSTM is trained with RMSprop optimizer and ExponentialLR scheduler.

The *UE-TP-rApp* is a regression task with input either an instance or a sequence of instances of 6 KPM metrics (i.e., *RSRP, RSRQ, SINR, PDSCH PRBs, PSDCH MCS, PUSCH MCS*), and predict *DL-throughput*. Both DNN and LSTM are trained using mean squared error (MSE) loss function, Adam optimizer and MultiStepLR scheduler.

The *Cell-TP-rApp* aggregates the predicted throughput results of *UE-TP-rApp* of all UEs belonging to the cell. So, the cell-TP-rApp queries BBU to get a list of UEs, then sends a series of requests to UE-TP-rApp with the UE-id to predict UE's throughput. In our implementation, UE-TP-rApp writes the predicted throughputs of UEs in InfluxDB. So, the Cell-TP-rApp just queries results from the database instead of requesting UE-TP-rApp to run inference again.

Table 2
Different ML-based models for Localization, UE's throughput prediction (UE-TP), Interference Management (IM) and Predictive maintenance (PM) rApps in Non-RT RIC with Instance-based (Inst.) or sequence-based (Seq.) inputs.

rApp	Input	ML Models
Localization	Inst.	Linear Classifier, XGBoost, Random Forest (RF), SVM, DNN(1x7,30,3)
	Seq.	Linear Classifier, RF, XGBoost, DNN(10x7,30,3), LSTM(50,50,50) | FC(50,3)
UE-TP	Inst.	Linear Regression, RF, SVM, DNN(1x6,10,1), XGBoost
	Seq.	Linear Regression, RF, XGBoost, DNN(10x6,10,1), LSTM(50,50,50) | FC(50,1)
IM	Seq.	DNN(1x7,20,1), Linear Classifier, RF XGBoost, LSTM(50,50,50) | FC(50,1)
PM	Inst.	DNN(1x7,20,1), Linear Classifier, RF, SVM, XGBoost
	Seq.	Linear Classifier, RF, XGBoost, DNN(10x7,20,1), LSTM(50,50,50) | FC(50,1)

LSTM(a, b)|FC(c, d) denotes an LSTM model with a stack of layers of a hidden units, and b hidden units; the output of LSTM is fed into fully connected layers of c and d units.

The *ES-rApp* sends control signals via the O1 interface to RUs (i) to turn off some RUs during night-time, and weekends; and (ii) reduce the TX-power of RUs that is underutilized. As our datasets were collected only for short periods of time, we could not build ML models to learn the traffic demand pattern of the cell/RUs in a daily or weekly manner. In this work, we use a common knowledge of working hours to define *energy-saving hours*, i.e., weekends, nighttime (8 pm–8 am), and public holidays. The energy-saving period is a pre-condition to reduce the cell capacity by turning off a number of active RUs, and reducing the TX-power of the remaining active RUs. After the pre-condition is matched, the ES-rApp checks if there is no associated UE under the cell for a consecutive 15 min before triggering to turn off some RUs while keeping a list of good coverage RUs (which are defined based on our deployment knowledge). If there is an active UE within the cell, the ES-rApp will sleep for an hour and before scanning for active UEs again. For example, in the outdoor cell, we turn off 1 RRU, and reduce the TX-power of the remaining RRU from 37 dBm to 14 dBm[1] during energy-saving hours. And in the indoor cell, we turn off 4 RRUs, and reduce the TX-power of the remaining RRU from 24 dBm to 8 dBm[1].

The *IM-rApp* is a classifier ML model that receives all 7 KPM metrics and outputs a binary value (i.e., 0—non-interference, 1—under interference). We construct this model in two stages: (i) check the floor of UE based on an output of the Localization-rApp, and (ii) design different classifier models for each floor because the signal under the interference zone of one floor is very similar to the poor signal area of the other floor. Both DNN and LSTM models are trained using binary cross-entropy loss function, Adam optimizer and MultiStepLR scheduler.

[1] This numbers are based on the range of allowed TX-power of RRUs while the radio is still active.

Table 3
Performance results of *Localization-rApp* with different ML models.

Input	Model	Acc (%)	Prec	Rec	F1	Size (kB)	Inf (µs)
Inst.	DNN	83.4	0.841	0.838	0.836	1.7	**2.1**
	Linear	80.2	0.836	0.801	0.805	**1.0**	3.5
	RF	82.8	0.827	0.831	0.828	1423.4	9.8
	SVM	82.5	**0.857**	0.825	0.829	177.6	112.7
	XGBoost	**84.4.**	0.845	**0.849**	**0.843**	973.4	**2.1**
Seq.	DNN	86.2	0.871	0.863	0.865	8.5	3.5
	Linear	80.0	0.820	0.801	0.805	**2.5**	4.9
	LSTM	**91.1**	**0.912**	**0.913**	**0.912**	201.0	27.4
	RF	85.8	0.857	0.860	0.858	1542.5	11.2
	XGBoost	87.4	0.880	0.878	0.872	792.0	**2.6**

The *PM-rApp* is a classifier ML model that receives all 7 KPM metrics and outputs a number of active RRUs, e.g., 1 or 2 active RRUs in our outdoor cell. If it is not the same as the number of RRUs that we get from the EMS system (which is 2 RRUs in our outdoor deployment scenario), the rApp will send an alert to the maintenance team for further analysis of why the actual measurement signals are only 1 active RRU. Noted that the PM-rApp is working when the ES-rApp does not trigger to turn off or reduce the TX-power of any RRUs. Both DNN and LSTM are trained using binary cross-entropy loss function, Adam optimizer and MultiStepLR scheduler.

6.1.3. Experimental results

Localization-rApp: Table 3 shows experimental results of localization-rApp with different ML-based models. We can see that other than the linear classifier, the sequence-based ML models always outperform the instance-based ML models. For the task of estimating the location of the UE, the LSTM model achieves an accuracy (Acc) of 91.1% and is complemented by decent performance in precision (Prec), recall (Rec) and F1-score. Even though the inference time of LSTM model is 27.4 µs which is the highest among the sequence-based ML models, it is acceptable to the requirements of non-realtime applications.

UE-TP-rApp: Predict traffic throughput of UE Table 4 shows the performance results of UE-TP-rApp under different ML models and with instance-based or sequence-based inputs. As a regression task, we use the R^2 score which is defined as

$$R^2 = 1 - \frac{\sum_{i=1}^{N}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{N}(y_i - \bar{y})^2},$$

where $\hat{y}_i$ is predicted throughput, y_i is the ground-true throughput of instance/sequence i, $\bar{y}$ is the mean throughput of the all N samples. Similar to the Localization-rApp results, we can observe the sequence-based ML models outperform the instance-based input ML models by a large margin. Among sequence-based ML models, the DL models (i.e., DNN, and

Table 4
Performance results of UE-TP-rApp with different ML models.

Input	Model	R^2	MSE	Size (kB)	Inf. (µs)
Inst.	DNN	**0.84**	**0.006**	**0.5**	0.7
	Linear	0.83	**0.006**	0.6	**0.4**
	RF	0.82	0.007	461.9	3.4
	SVM	0.82	0.007	115.6	24.4
	XGBoost	0.72	0.010	537.8	0.8
Seq.	**DNN**	**0.90**	**0.004**	2.4	**0.4**
	Linear	0.88	**0.004**	**1.4**	3.7
	LSTM	**0.90**	**0.004**	200.1	21.7
	RF	**0.90**	**0.004**	460.4	4.1
	XGBoost	0.85	0.005	554.4	1.3

LSTM) outperform the classical ML models other than RF with R^2 of 0.90. Considering other metrics, we choose DNN model to deploy in our final experiment because it has a lower MSE, and compact model size, and a faster inference time (i.e., 10 times faster than RF model and 54 times faster than the LSTM model).

ES-rApp: Table 5 shows the comparison of energy consumption of the indoor and outdoor cells with and without enabling the ES-rApp, for a period of 1 year. The estimation is based on the gain analysis calculations by O-RAN [37] and the RAN vendor specification [38]. The energy consumption E of a cell (with a number $\#RRU$) over a period t can be calculated as

$$E\ [\text{Wh}] = P_{RRU}\ [\text{W}] \times t\ [\text{h}] \times \#RRU$$

Based on the design in Section 6.1.2, considering a typical year with 52 weeks (i.e., 8736 hours per year), and assuming 7 days of public holidays falling on weekdays per year. Based on this simplified calendar, we have 108 energy-saving hours a week (i.e. $5 \times 12 + 24 * 2$ h). If a public holiday falls on a weekday, we can save an extra 12 h of public holiday for that day.[2] Therefore, every year, we have 5700 energy-saving hours, i.e., 108×52 h and 7 public holidays on weekdays (7×12 h). There are 3036 h of peak-hours per year ($(52 \times 5 - 7) \times 12$) where we do not enable energy-saving.

For our deployment, we have 5 indoor RRUs and 2 outdoor RRUs. The TX-power of an indoor RRU is 1 W (i.e., 4×250 mW), while the TX-power of an outdoor RRU is 20 W (i.e., 4×5 W). During energy-saving hours, most of RRUs are turned off, and only one coverage RRU is configured with reduced TX-power. The reduced TX-power for the indoor RRU is 0.025 W (i.e., 4×8 dBm), and for outdoor RRU, it is 0.1 W (i.e., 4×14 dBm).

As shown in Table 5, deploying the ES-rApp in our environment would save us almost 65% of the TX-energy. And the process is fully automated by the ES-rApp. We can further improve the ES-rApp model as more data become available. Enormous energy savings can be achieved when the ES-rApp is deployed on a larger scale.

[2] We assume the deployment in urban central, e.g., offices, where no 5G usage during holidays.

Table 5
Comparison TX-energy consumption (Wh) of the indoor and outdoor cells with/without Energy-Saving ES-rApp for a period of 1 year.

Cell (#RRU)	w/o ES-rApp	w/ ES-rApp	Saving (%)
Indoor (5)	43,680	15,322	28,357 (64.9)
Outdoor (2)	349,440	122,010	227,430 (65.1)

Table 6
Performance results of Interference Management IM-rApp with different ML models.

Input	Model	Acc (%)	Prec	Rec	F1	Size (kB)	Inf. (µs)
Seq	DNN	88.8	0.645	0.537	0.580	5.5	2.2
	Linear	90.0	0.923	0.612	0.695	**1.4**	**0.1**
	LSTM	**90.1**	0.854	**0.804**	**0.828**	200.8	14.0
	RF	87.8	**0.989**	0.623	0.736	647.7	24.5
	XGBoost	89.5	0.962	0.714	0.802	99.0	1.1

Table 7
Performance results of Predictive Maintenance PM-rApp with different ML models.

Input	Model	Acc (%)	Prec	Rec	F1	Size (kB)	Inf. (µs)
Inst.	DNN	87.7	0.815	0.966	0.884	1.0	2.3
	Linear	89.3	0.821	**1.000**	0.901	**0.9**	**0.7**
	RF	89.5	0.885	0.905	0.895	2466.9	8.3
	SVM	90.8	0.845	0.997	0.914	130.3	23.7
	XGBoost	**92.6**	**0.923**	0.927	**0.925**	297.5	1.5
Seq.	**DNN**	**99.1**	**0.985**	0.997	**0.991**	5.5	3.8
	Linear	94.6	0.901	**1.000**	0.948	**1.4**	9.4
	LSTM	97.3	0.948	**1.000**	0.973	200.8	29.1
	RF	97.5	0.961	0.989	0.975	2060.1	15.4
	XGBoost	96.2	0.952	0.970	0.961	247.6	**2.3**

IM-rApp: Table 6 shows average performance results of Inference Management IM-rApp using different ML models with only sequence-based input. Note that this IM-rApp has two stages as presented in Section 6.1.2, the results are average performance results of three classifiers for each floor. In this rApp, it can be noticed that DL models outperform the classical ML models. Between the two DL models, the LSTM model achieves the highest accuracy of 90.1%, F1-score of 0.828. Even though its inference time is the second highest, which is 14 µs, it still complies with the time requirement of non-realtime applications.

PM-rApp: Predictive Maintenance Application Table 7 shows performance results of Predictive Maintenance PM-rApp using different ML models with instance-based and sequence-based input. Similar to the above two rApps, we can see the sequence-based ML models always outperform the instance-based ML models by a large margin. In this rApp, the DNN model achieves an impressive accuracy of 99.1%. This accuracy is accompanied by high precision, recall, and F1-score. Although the inference time of DNN model is not the shortest, it is still faster than RF and LSTM (i.e., 4 times faster than RF model and 7 times faster than LSTM model).

It is noteworthy that for all the rApps, the trained models exhibit inference latency below 1 s, meeting the time requirements of an rApp. Moreover, the choice of input representation

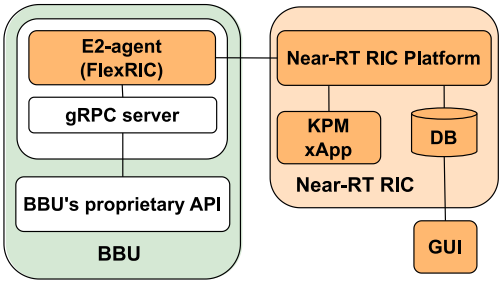


Fig. 8. Near-RT RIC implementation architecture to convert commercial 5G RAN to O-RAN compliant E2 interface.

is observed to have a significant impact on model performance. Models with sequential-based input consistently demonstrate superior performance compared to those relying on instance-based input, as indicated by the results presented in Tables 3, 4, and 7. Also, DL models always outperform classical ML model in each rApp.

6.2. Near-RT RIC applications in a private 5G network

6.2.1. Experimental setup and implementation

Our setup has a complete QCT 5G RAN system, which includes two BBUs (1 for outdoor cell, 1 for indoor cell), 2 outdoor RRUs, and 5 indoor RRUs. The UE can be a commercial phone or drone with a 5G dongle.

We write a wrapper to adapt vendor-specific API to support E2 interface routines and communicate with FlexRIC. This helps us leverage the commercial system to collect data and use near-RT RIC open-source work to study the data. We use docker to deploy two wrapper services in the BBU so that our new services do not change the environment of the BBU. The first service is "E2-agent" adapting from FlexRIC receives E2 messages and handles E2 service routines. It uses gRPC to communicate with the second service, which is a gRPC server that will handle requests and call the proprietary API of the BBU to get the information. We split into two services because if later we have another BBU with a different set of proprietary APIs, we can reuse the E2 agent service and only need to write a new service with a new set of APIs. Although split into two, we still deploy in only one container.

6.2.2. Demo

In the demo, we use a 5G Samsung S21 phone as a UE and observe the KPM metrics that are collected by KPM-xApp. As shown in Fig. 8, the GUI shows the KPIs of the connection to the phone. In our implementation, we can collect UE-level KPI and cell-level KPI such as DL/UL throughput in MAC or RLC layer; number of DL/UL resource blocks used per UE per second; number of UL/DL timeslots granted for the UE per second; average MCS, DL/UL layer, CQI (Channel Quality Indicator), RSSI, BLER (Block Error Rate) for each UE. We can query the data with an interval of 1 s. Fig. 9 shows an example of KPM metrics in the video demo.[3] The shown in the figures, the KPIs are collected periodically with 1 1-s duration and change lively when the UE surfs the internet.

[3] The video demo is available at: https://tinyurl.com/RICKPMPrivate5G.

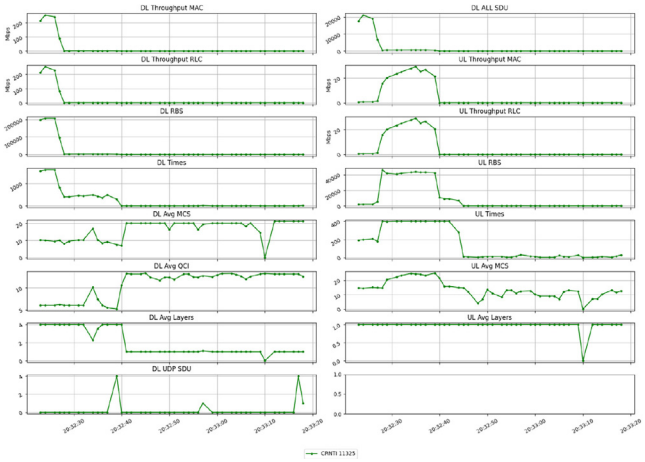


Fig. 9. UE's performance metrics from KPM-xApp.

7. Conclusions

In this paper, we presented open-source projects for Non-RT and Near-RT RIC, and their pros and cons. Based on the open-source Non-RT RIC, we designed a suite of rApps for several applications (i.e., energy efficiency, interference management, and predictive maintenance) and evaluated them on a real-world dataset of the actual private 5G network testbed in SUTD, Singapore. The experimental results showed the ML-based models with sequence input outperform the similar model with instance input, and the best ML models achieve more than 90% of accuracy for all applications with inference time less than 30 μs. Besides Non-RT RIC, we also presented how we can adopt an open-source project to add an E2 agent in a commercial 5G RAN whose E2 interface is not ready yet.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Research Foundation, Singapore and Infocomm Media Development Authority under its Future Communications Research & Development Programme (FCP); in part by the Ministry of Science and ICT (MSIT), South Korea, through the Information Technology Research Center (ITRC) Support Program, supervised by the Institute of Information & Communications Technology Planning & Evaluation (IITP), under Grant IITP-2021-0-02046; and in part by IITP grant funded by the Korea government(MSIT) (No. RS-2022-00207387, Development of Intelligent Network Slicing Technology based on 5G Open RAN).

References

[1] O-RAN alliance, 2023, Accessed: 7 Nov 2023, https://www.o-ran.org/.

[2] Juniper RAN intelligent controller (RIC), 2023, Accessed: 16 Nov 2023, https://www.juniper.net/us/en/products/network-automation/ran-intelligent-controller.html.

[3] VMWare RIC, 2023, Accessed: 16 Nov 2023, https://telco.vmware.com/products/ric.html.

[4] Accessed: 16 Nov 2023 (2023), https://www.capgemini.com/wp-content/uploads/2023/06/RIC_brochure_Jan23-2.pdf.

[5] D. Johnson, D. Maas, J. Van Der Merwe, Nexran: Closed-loop RAN slicing in POWDER -a top-to-bottom open-source open-RAN use case, in: Proc. of the 15th ACM Workshop on WiNTECH, WiNTECH '21, ACM, New York, NY, USA, 2021, pp. 17–23.

[6] L. Bonati, S. D'Oro, M. Polese, S. Basagni, T. Melodia, Intelligence and learning in O-RAN for data-driven nextg cellular networks, IEEE Commun. Mag. 59 (10) (2021) 21–27.

[7] H.-M. Yoo, J.-S. Rhee, S.-Y. Bang, E.-K. Hong, Load balancing algorithm running on open RAN ric, in: 2022 13th International Conference on Information and Communication Technology Convergence, ICTC, 2022, pp. 1226–1228.

[8] M. Hoffmann, S. Janji, A. Samorzewski, Ł. Kułacz, C. Adamczyk, M. Dryjański, P. Kryszkiewicz, A. Kliks, H. Bogucka, Open RAN xapps design and evaluation: Lessons learnt and identified challenges, IEEE J. Sel. Areas Commun. (2023).

[9] O-RAN.WG1, O-RAN Work Group 1 (Use Cases and Overall Architecture) Use Cases Detailed Specification, Technical report (tr), O-RAN Alliance, 2023, R003-v12.00.

[10] O.-R. Alliance, White paper: O-RAN: Towards an open and smart RAN, 2023, Accessed 22 Nov 2023.

[11] OpenAirInterface, Accessed: 16 Nov 2023, https://openairinterface.org/.

[12] M. Polese, L. Bonati, S. D'Oro, S. Basagni, T. Melodia, Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges, IEEE Commun. Surv. Tutor. 25 (2) (2023) 1376–1411.

[13] SRSRAN, SRSRAN—Open source 4G software radio, 2022, Accessed 16 Nov 2023, https://github.com/srsran/srsRAN_4G.

[14] O-RAN.WG1, O-RAN Work Group 1 (Use Cases and Overall Architecture) Use Cases Analysis Report, Technical report (tr), O-RAN Alliance, 2023, R003-v12.00.

[15] O-RAN software community documents, 2023, Accessed: 7 Nov 2023, https://docs.o-ran-sc.org/en/latest/index.html.

[16] SD-RAN, Sdran.docs, 2023, Accessed: 7 Nov 2023, https://docs.sd-ran.org/master/introduction.html.

[17] F.A. Bimo, F. Feliana, S.-H. Liao, C.-W. Lin, D.F. Kinsey, J. Li, R. Jana, R. Wright, R.-G. Cheng, OSC community lab: The integration test bed for O-RAN software community, in: 2022 IEEE Future Networks World Forum, FNWF, 2022, pp. 513–518, http://dx.doi.org/10.1109/FNWF55208.2022.00096.

[18] OAI, Flexric.gitlab, 2023, Accessed: 7 Nov 2023, https://gitlab.eurecom.fr/mosaic5g/flexric.

[19] R. Schmidt, M. Irazabal, N. Nikaein, FlexRIC: An SDK for next-Generation SD-RANs, CoNEXT '21, Association for Computing Machinery, New York, NY, USA, 2021, pp. 411–425, http://dx.doi.org/10.1145/3485983.3494870.

[20] O-RAN software community non-RT RIC documents, 2023, Accessed: 20 Nov 2023, https://docs.o-ran-sc.org/projects/o-ran-sc-nonrtric/en/latest/overview.html.

[21] O-RAN.WG2.A1.TP, O-RAN A1 interface: Transport Protocol 3.0, Technical report (tr), O-RAN Alliance, 2023, R003-v3.00.

[22] O-RAN.WG10, O-RAN Operations and Maintenance Interface Specification 11.0, Technical report (tr), O-RAN Alliance, 2023, R003-v11.00.

[23] 3GPP, 5G; Management and orchestration; Generic management services (3GPP TS 28.532 version 16.4.0 Release 16), 2020, URL https://www.etsi.org/deliver/etsi_ts/128500_128599/128532/16.04.00_60/ts_128532v160400p.pdf, Accessed: 22 Nov 2023.

[24] O-RAN Software Community, O-RAN software community wiki, 2023, URL https://wiki.o-ran-sc.org/pages/viewpage.action?pageId=3604819, Accessed: 2023-11-20.

[25] O.-R. Alliance, O-RAN working group 3, near-RT RIC architecture, 2023, Published: 2023.10.

[26] Flexric.wiki.e42, 2023, Accessed: 8 Nov 2023, https://gitlab.eurecom.fr/mosaic5g/flexric/-/wikis/What-is-FlexRIC.

[27] SWIG (simplified wrapper and interface generator), 2023, Accessed: 7 Nov 2023, https://swig.org/svn.html.

[28] SD-RAN, RAN simulator, 2023, Accessed: 20 Nov 2023, https://docs.sd-ran.org/master/ran-simulator/README.html.

[29] Flexric.gitlab, 2023, Accessed: 7 Nov 2023, https://gitlab.eurecom.fr/mosaic5g/flexric/-/tree/master/src/agent?ref_type=heads.

[30] O.-R. Alliance, O-RAN working group 3, near-real-time RAN intelligent controller, E2 application protocol (E2AP), 2022, Published: 2022.02.07.

[31] M. Polese, L. Bonati, S. D'Oro, S. Basagni, T. Melodia, Colo-RAN: Developing machine learning-based xapps for open RAN closed-loop control on programmable experimental platforms, IEEE Trans. Mob. Comput. 22 (10) (2023) 5787–5800, http://dx.doi.org/10.1109/TMC.2022.3188013.

[32] D. Boukhayar Bouazza, Analysis of AI/ML algorithms for the management of open 5G mobile networks: xapps in O-RAN, 2022, https://riunet.upv.es/handle/10251/186962.

[33] I. Chatzistefanidis, N. Makris, V. Passas, T. Korakis, ML-based traffic steering for heterogeneous ultra-dense beyond-5g networks, in: 2023 IEEE Wireless Communications and Networking Conference, WCNC, 2023, pp. 1–6, http://dx.doi.org/10.1109/WCNC55385.2023.10118923.

[34] F. Kavehmadavani, V.-D. Nguyen, T.X. Vu, S. Chatzinotas, Intelligent traffic steering in beyond 5G open RAN based on lstm traffic prediction, IEEE Trans. Wireless Commun. 22 (11) (2023) 7727–7742, http://dx.doi.org/10.1109/TWC.2023.3254903.

[35] O.-R. Alliance, O-RAN working group 3, E2 general aspects and principles, 2023, Published: 2023.10.

[36] O.-R.S. Community, Anomaly detection (AD) xapp, 2020, Accessed 22 Nov 2023, https://github.com/o-ran-sc/ric-app-ad.

[37] O-RAN.WG1, O-RAN Work Group 1 (Use Cases and Overall Architecture) Network Energy Saving Use Cases Technical Report, Technical report (tr), O-RAN Alliance, 2023, R003-v02.00.

[38] Quanta Cloud Technology (QCT), QCT 5G RAN omniran solution, 2023, Accessed 21 Nov 2023, https://go.qct.io/telco/omnipod-enterprise-5g/qct-5g-ran/.